

IEOR 4101: Probability, Statistics and Simulation

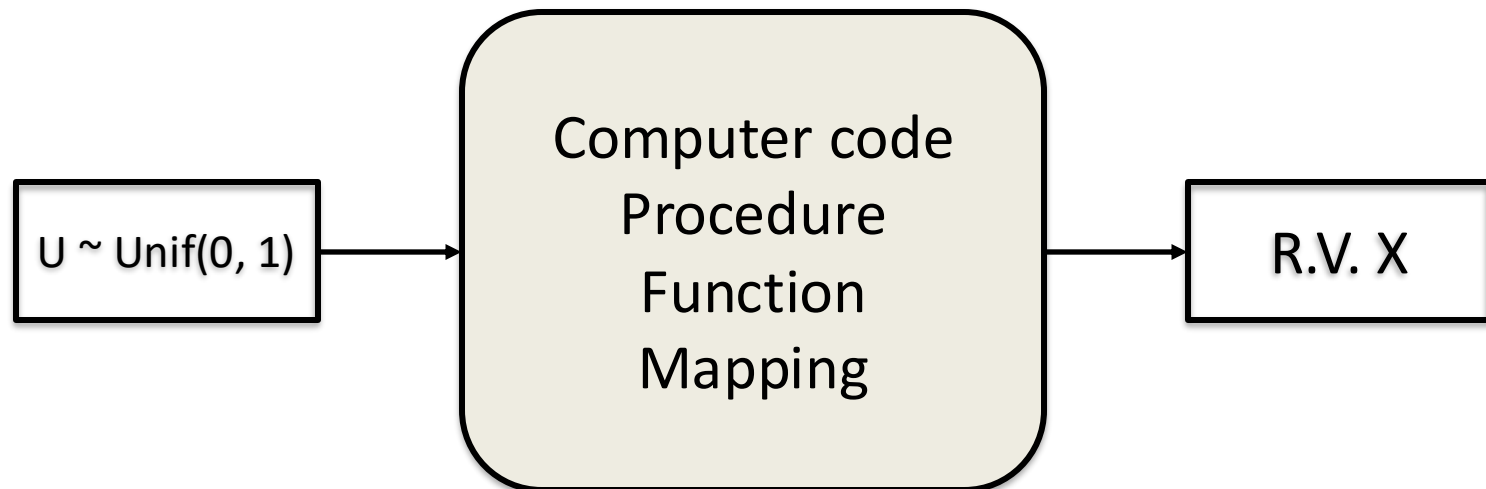
Lecture 10: Simulation

What is Simulation

- A computational tool for stochastic models with no closed-form
- Requires:
 - Replicate random scenarios according to prescribed random variables
 - Statistical output analysis

A High-Level View of RV Simulation

- What does it mean to generate a random variable?
- In computing languages, $\text{Unif}(0, 1)$ is often taken as the basis for generating other random variables; e.g., $\text{RAND}()$ in Excel; rand in Matlab; etc
- To simulate a random variable X , the main task is to design a procedure that takes $U \sim \text{Unif}(0, 1)$ as input and converts it into the random variable X



The Inverse Transform Method

- Example: Pseudocode to generate Bernoulli rv X with parameter p :

$U = \text{rand};$ ← U is generated from $\text{Unif}(0, 1)$
if $U \leq p$
 return $X = 1$;
else return $X = 0$

- To generate a rv X , we need
 - A primitive to generate $U \sim \text{Unif}(0, 1)$ (this is the only “random” part)
 - A procedure to convert U into X
 - This “procedure” is deterministic; can view it as a function/mapping

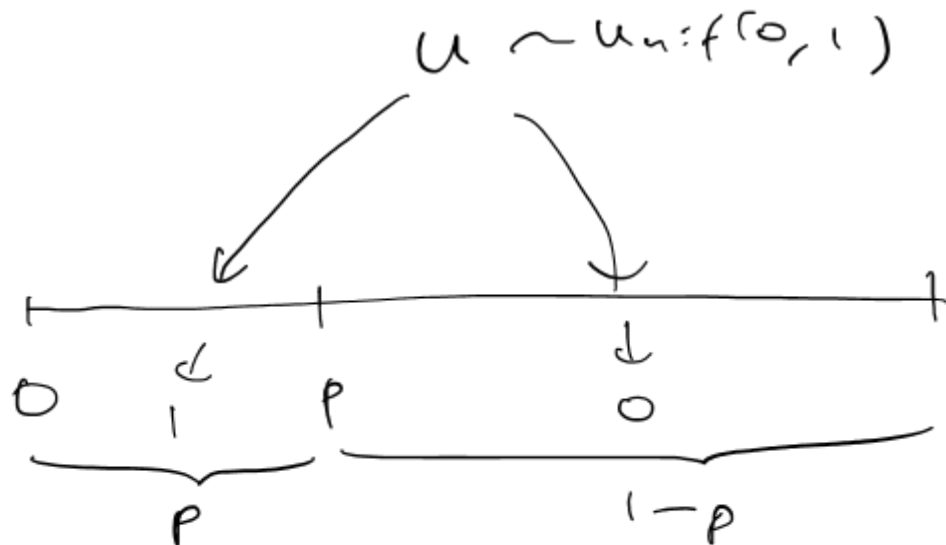
To argue the pseudocode is outputting
 $\text{Ber}(p)$,

possible values of the output = 0 or 1.

$$\begin{aligned} P(\text{output} = 1) &= P(U \leq p) \quad U \sim \text{unif}(0, 1) \\ &= \int_0^p 1 \, dx \\ &= p \end{aligned}$$

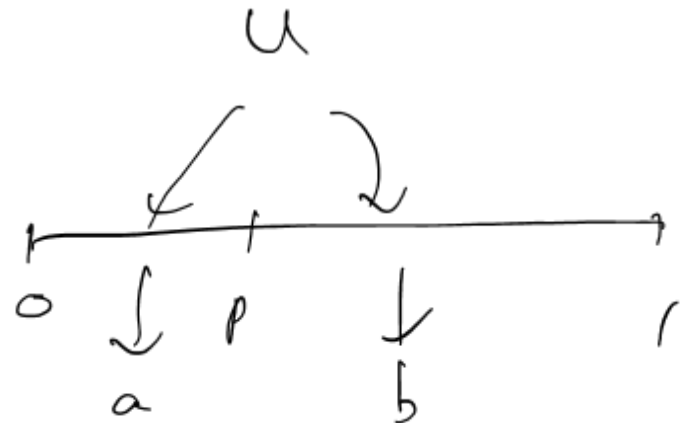
The Inverse Transform Method

- Pseudocode to generate Bernoulli rv X with parameter p :
 $U = \text{rand};$
 if $U \leq p$
 return $X = 1;$
 else return $X = 0$
- Idea: divide the interval $[0, 1]$ up into two pieces $[0, p]$ and $[p, 1]$



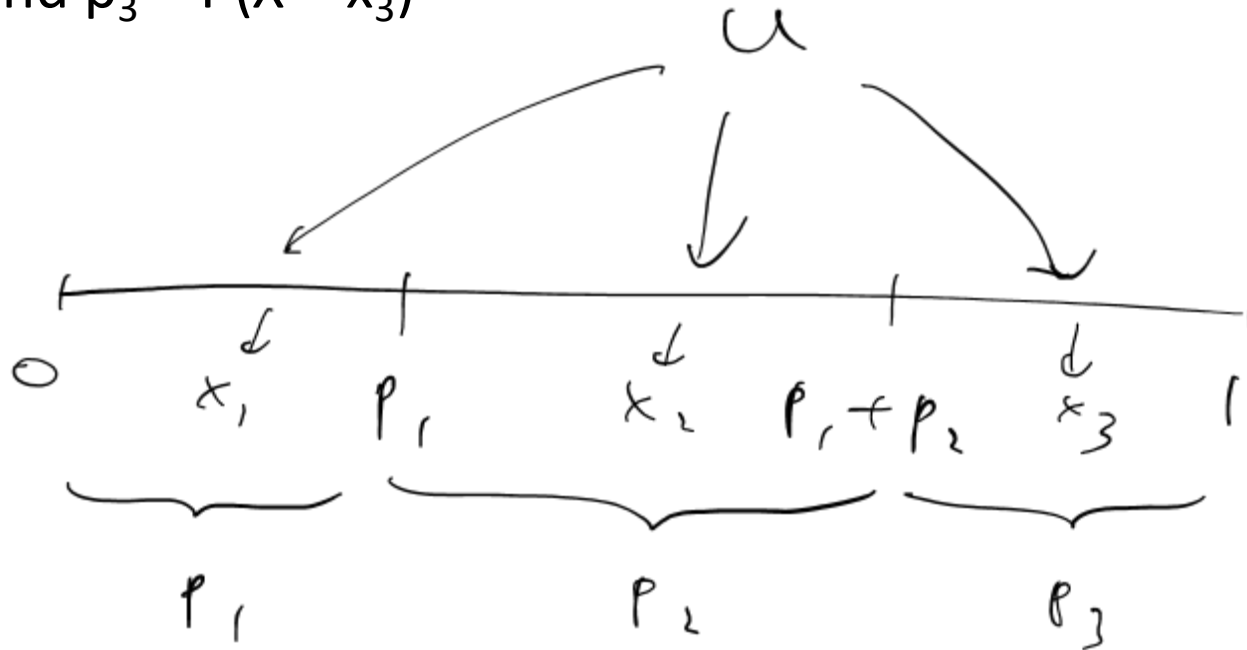
The Inverse Transform Method

- Nothing special about X taking values 0 and 1
- Suppose X is a rv that has the pmf: $P(X=a) = p$; $P(X=b) = 1-p$
- Pseudocode to generate rv X :
 $U = \text{rand};$
 if $U \leq p$
 return $X = a$;
 else return $X = b$



The Inverse Transform Method

- Suppose we wish to generate a discrete random variable X taking three values x_1, x_2, x_3 , with pmf given by $p_1 = P(X = x_1)$, $p_2 = P(X = x_2)$ and $p_3 = P(X = x_3)$

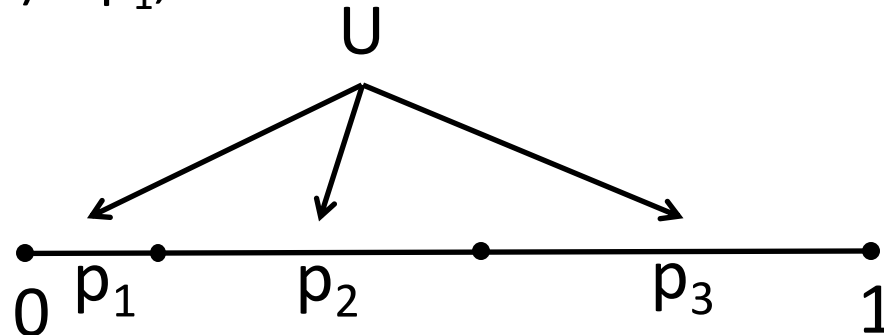


e.g. $x_1 = 0, x_2 = 1.5, x_3 = 2.5$

$$p_1 = \frac{1}{2}, p_2 = \frac{1}{3}, p_3 = \frac{1}{6}$$

The Inverse Transform Method

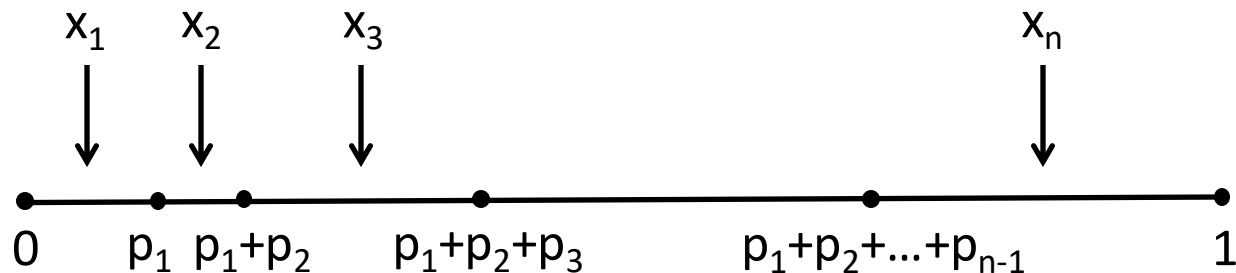
- Suppose we wish to generate a discrete random variable X taking three values x_1, x_2, x_3 , with pmf given by $p_1 = P(X = x_1)$, $p_2 = P(X = x_2)$ and $p_3 = P(X = x_3)$
- Can chop up the interval $[0, 1]$ into three pieces
- $P(U \text{ in first piece}) = p_1$, etc



- Pseudocode to generate X :
 $U = \text{rand};$
 if $0 \leq U \leq p_1$, return $X = x_1$;
 if $p_1 < U \leq p_1 + p_2$, return $X = x_2$;
 else return $X = x_3$

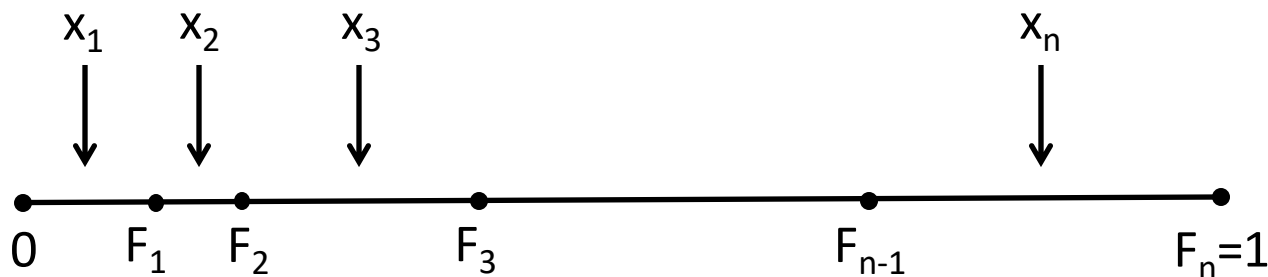
The Inverse Transform Method

- Can generalize the idea even further
- Suppose X is a general discrete random variable with pmf $p_k = P(X = x_k)$, $k = 1, 2, \dots, n$
- Suppose $x_1 < x_2 < \dots < x_n$
- Divide the interval $[0, 1]$ into n subintervals of lengths $p_1, p_2, \dots, p_n$ respectively
- Output $X = x_k$ if U lies within the k^{th} subinterval



The Inverse Transform Method

- More formally, define $F_k = p_1 + p_2 + \dots + p_k$, $k = 1, 2, \dots, n$
- F_k are the end points of the subinterval
 - They are also the “jump” values of the cdf
- Output $X = x_k$ if U lies in (F_{k-1}, F_k) , where $F_0 = 0$

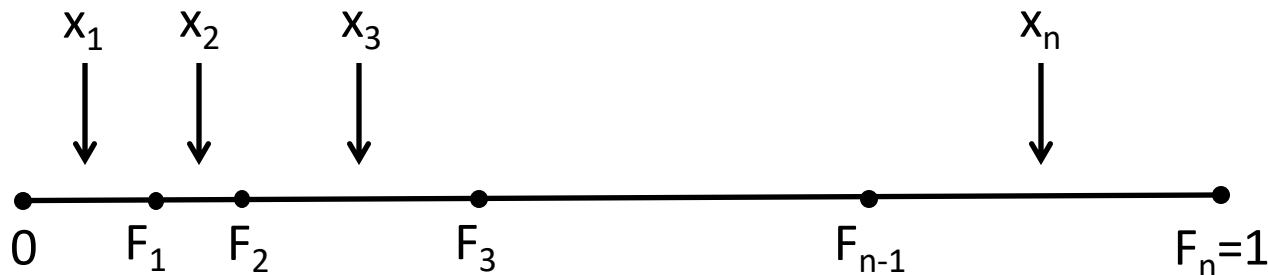


The Inverse Transform Method

- More formally, define $F_k = p_1 + p_2 + \dots + p_k$, $k = 1, 2, \dots, n$
- F_k are the end points of the subinterval
 - They are also the “jump” values of the cdf
- Output $X = x_k$ if U lies in (F_{k-1}, F_k) , where $F_0 = 0$
- Pseudocode to generate X :

- $U \sim \text{Unif}(0, 1)$; Set $F_0 = 0$ ← initialization
- $k = 0$;
- while $U > F_k$
- $k = k + 1$;
- output $X = x_k$

$F_0 = 0$
 $k = 0$
 $U > F_0 = 0 \Rightarrow \checkmark$
 $k = 1$
 $U > F_1 \Rightarrow$ if true, continue
 $\Rightarrow$ if false, stop
 $\downarrow$
output x_1



Using "while" loop, we can

- save lines compared to "if"

- for v.v. that has infinite number of possible values, it's not possible to use "if".

The Inverse Transform Method

- The method applies the following function to $U \sim \text{Unif}(0, 1)$

$$G(u) = \begin{cases} x_1 & \text{if } u \in [0, F_1] \\ x_2 & \text{if } u \in (F_1, F_2] \\ \dots & \\ x_n & \text{if } u \in (F_{n-1}, 1] \end{cases}$$

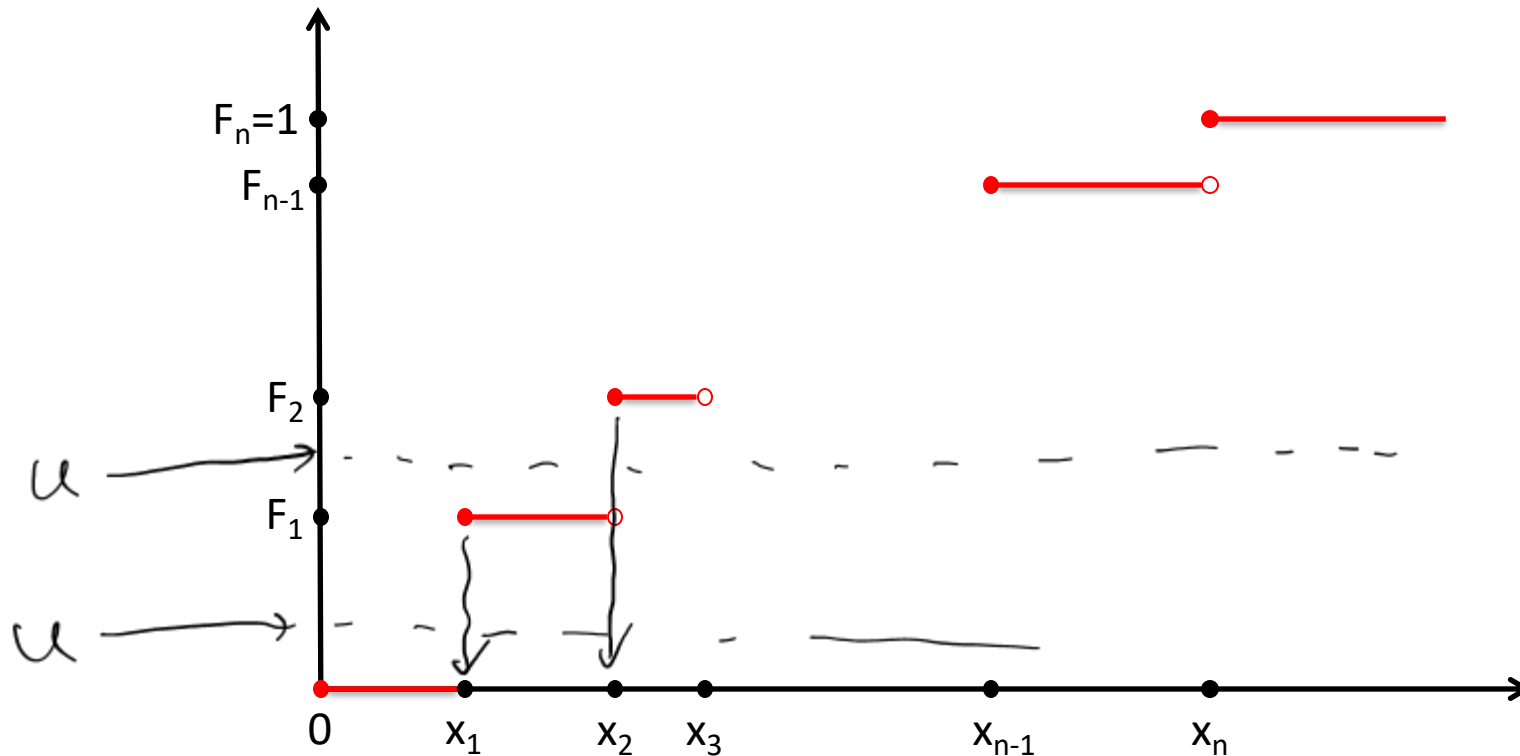
*Pseudo code in
previous slide:*

- Generate $U \sim \text{Unif}(0, 1)$*
- Output $G(u)$*

- What does the function G do?

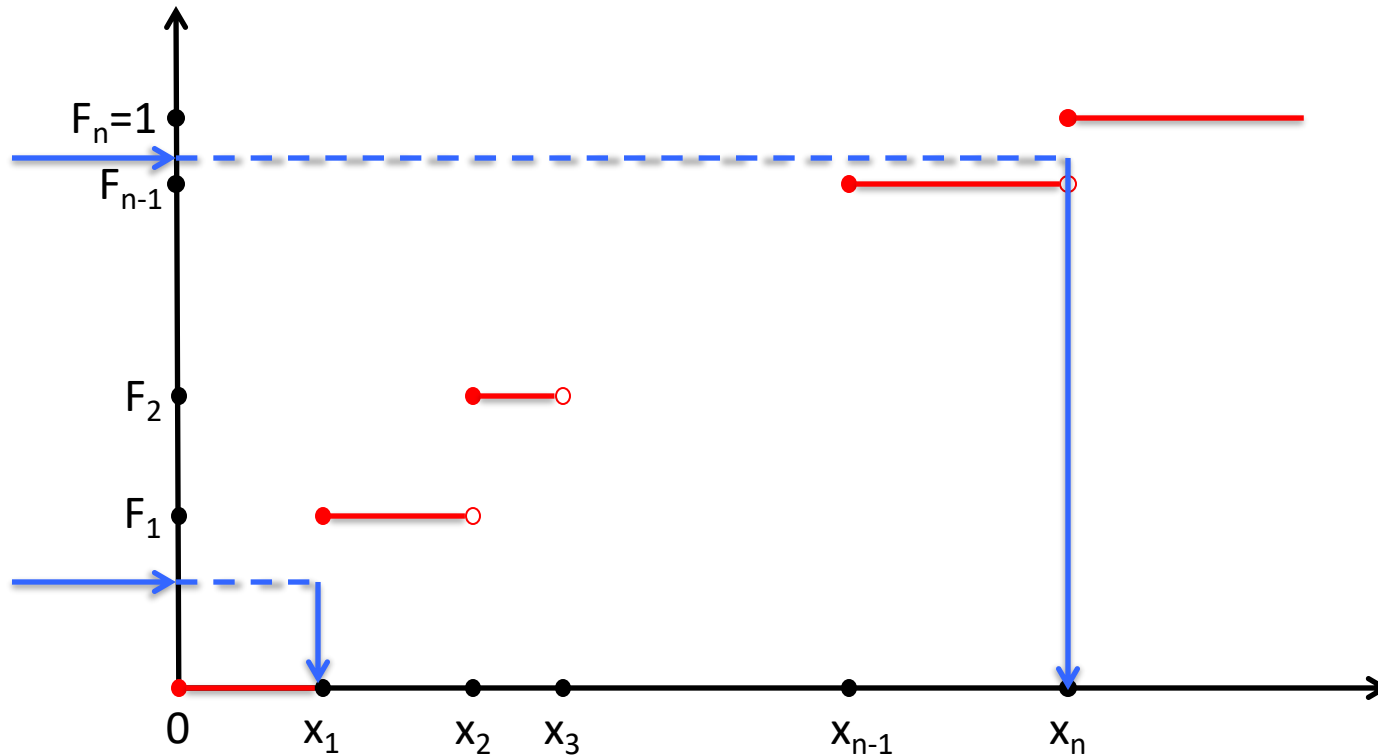
The Inverse Transform Method

- Plot of the cdf F looks like



The Inverse Transform Method

- This is what G does



- Hence the name “inverse transform”

G is the inverse of cdf.

Generating Binomial RV

- Let $X \sim \text{Bin}(n, p)$

Generating Binomial RV

- Let $X \sim \text{Bin}(n, p)$
- Calculate $p_k = \binom{n}{k} p^k (1-p)^{n-k}$ and $F_k = p_0 + p_1 + \dots + p_k$ for $k = 0, 1, \dots, n$, and $F_{-1} = 0$
- Alternately, use the recursion $F_{-1} = 0$ and $F_k = F_{k-1} + p_k$ for $k = 0, 1, 2, \dots, n$
- Pseudocode to generate X :
 - $U \sim \text{Unif}(0, 1)$; Set $F_{-1} = 0$
 - $k = -1$;
 - while $U > F_k$
 - $k = k+1$;
 - output $X = k$

$$F_{-1} = 0$$

$$k = -1$$

$$U > F_{-1} = 0 \Rightarrow \checkmark$$

$$k = 0$$

$$U > F_0 = p_0 \Rightarrow \begin{array}{l} \text{if true,} \\ \text{continue} \end{array}$$

$$\Rightarrow \text{if false,}$$

$$\text{output } X = 0$$

Generating Binomial RV

- Sometimes we can just use the interpretation of the rv to simulate it or combine it with the inverse transform method
- Let $X \sim \text{Bin}(n, p)$; use Bernoulli trial interpretation

- Pseudocode:

- $X = 0; I = 0;$

- for $i=1:n$

- $U = \text{rand};$

- if $U \leq p$

- $I = 1;$

- else $I = 0;$

- end

- $X = X + I;$

- end

successes

indicator of success

pseudo code for generating $\text{Ber}(p)$

- Which approach is preferable?

When p is small, 1st approach is preferable
because most likely X is small and
hence the # iterations in the "while"
loop is small.

When p is large, 2nd approach is preferable.

Generating Geometric RV

- Let $X \sim \text{Geo}(p)$, with pmf $P(X = k) = (1-p)^{k-1}p$, $k = 1, 2, \dots$
- Using the interpretation of geometric rv, pseudocode to simulate a $\text{Geo}(p)$ distribution:

```
I = 0;  indicator of success
X = 1;  count of trials
while I = 0
    U = rand;
    if U ≤ p
        I = 1;
    else X = X + 1;  increase count by 1
    end
end
end
Output X.
```

generator of $\text{Ber}(p)$ by ITM

Generating Geometric RV

- Use the inverse transform method

Generating Geometric RV

- Use the inverse transform method
 - $P(X = k) = p(1-p)^{k-1}, k = 1, 2, \dots$
 - $F_k = P(X = 1) + \dots + P(X = k) = 1 - P(X > k) = 1 - (1-p)^k$
 - Thus, generate $U \sim \text{Unif}(0, 1)$ and find k such that $F_{k-1} < U \leq F_k$
 - $X = \min\{k: U \leq F_k\} = \dots = \left\lceil \frac{\log(1-U)}{\log(1-p)} \right\rceil$
 - Simplify to get $\left\lceil \frac{\log(U)}{\log(1-p)} \right\rceil$
- Which approach is preferable?

1.7 means
"rounding up".

pmf of $\text{Geo}(p)$:

$$P(X=k) = (1-p)^{k-1} p, \quad k=1, 2, \dots$$

cdf of $\text{Geo}(p)$:

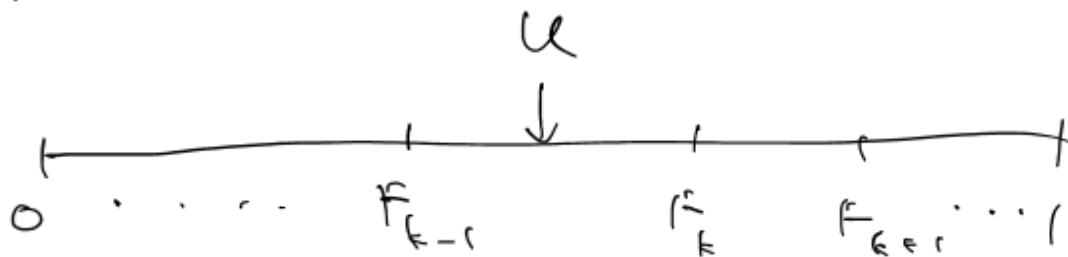
$$\begin{aligned} P(X \leq k) &= 1 - P(X > k) \\ &= 1 - (1-p)^k \end{aligned}$$

TM:

If $F_{k-1} \leq U < F_k$, output k

This is equivalent to

output $X = \min \{k : U \leq F_k\}$



Consider

$$X = \min \{ k : u \leq F_k \}$$

Here

$$u \leq F_k = 1 - (1-p)^k$$

$$\Rightarrow (1-p)^k \leq 1-u$$

$$\Rightarrow k \log(1-p) \leq \log(1-u)$$

$$\Rightarrow k \geq \frac{\log(1-u)}{\log(1-p)} \Rightarrow X = \left\lceil \frac{\log(1-u)}{\log(1-p)} \right\rceil$$

When p is small, the 1st approach (using the interpretation of Geo) might take a lot of iterations. $\Rightarrow$ 2nd approach is preferable as it only uses simple function evaluation

When p is large, 1st approach is preferable because the number of iterations is small, while 2nd approach has function evaluation.

Generating Poisson RV

- Let $X \sim \text{Poi}(\lambda)$
- pmf of X is $P(X = k) = p(k) = e^{-\lambda} \cdot \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$
- $F(k) = \sum_{i=0}^k p(i)$
- An alternate approach to compute $F(k)$:
 - $p(0) = e^{-\lambda}; F(0) = p(0)$
 - $p(k+1) = \frac{\lambda}{k+1} \cdot p(k)$
 - $F(k+1) = F(k) + p(k+1)$
- Apply the inverse transform method as before

} recursion.

Generating Poisson RV

- Pseudocode

$U = \text{rand};$

$p = \exp(-\lambda);$ $\leftarrow$ storage for pmf

$F = p;$ $\leftarrow$ storage for cdf

$i = 0;$

while $U > F$

$p = \lambda p / (i+1);$

$F = F + p;$

$i = i + 1;$

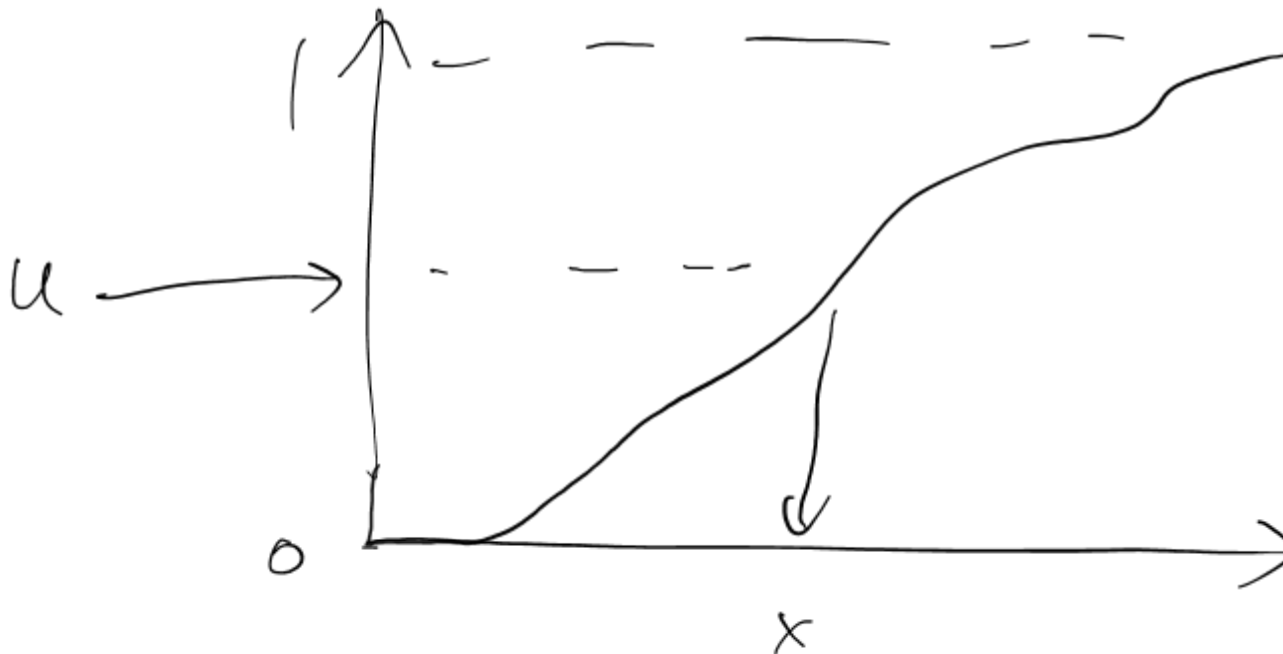
end while

output i

} use recursion for
pmf and cdf of
 $\text{Poi}(\lambda)$

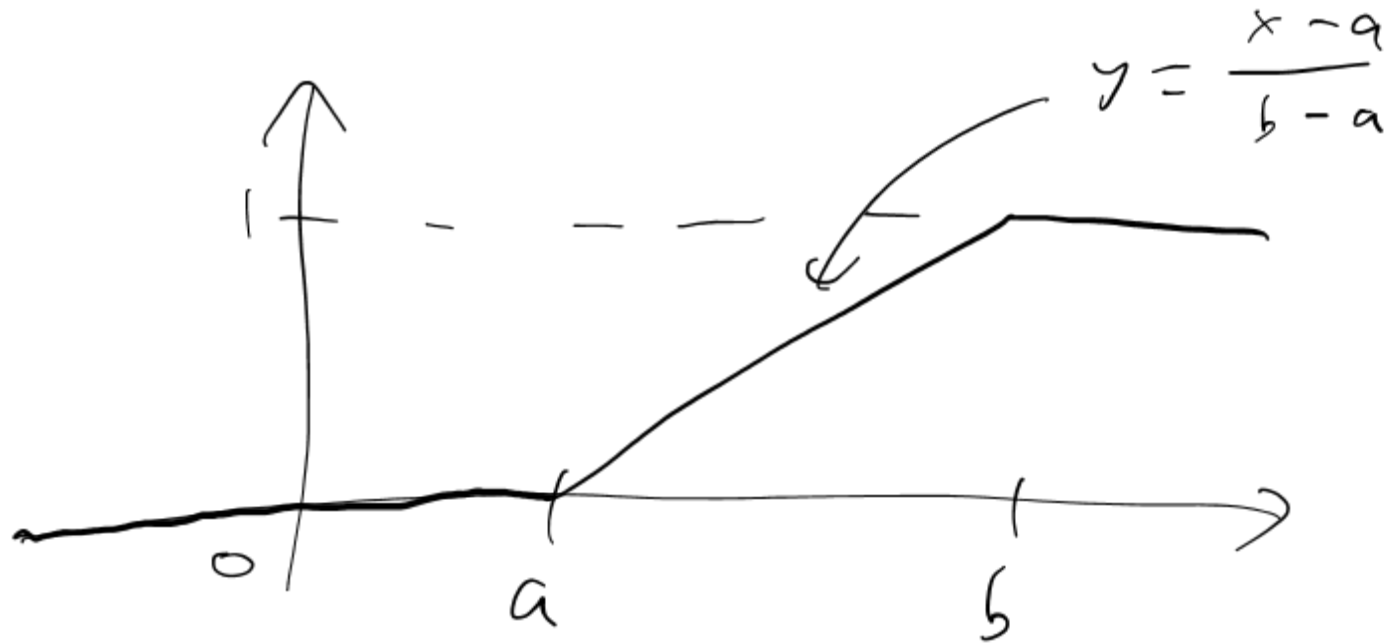
Inverse Transform Method for Continuous RV

- To generate a (continuous) random variable X with cdf $F(x)$:
 - Compute the inverse function $G(u)$
 - Generate $U \sim \text{Unif}(0, 1)$
 - Output $X = G(U)$



Generating Uniform RV

- Let $X \sim \text{Unif}(a, b)$
- If $a \leq x \leq b$, $P(X \leq x) = (x-a)/(b-a)$
- If $x < a$, $P(X \leq x) = 0$; if $x > b$, $P(X \leq x) = 1$



ITM:

$$u = \frac{x-a}{b-a} \Rightarrow x = a + (b-a)u$$

pseudo-code:

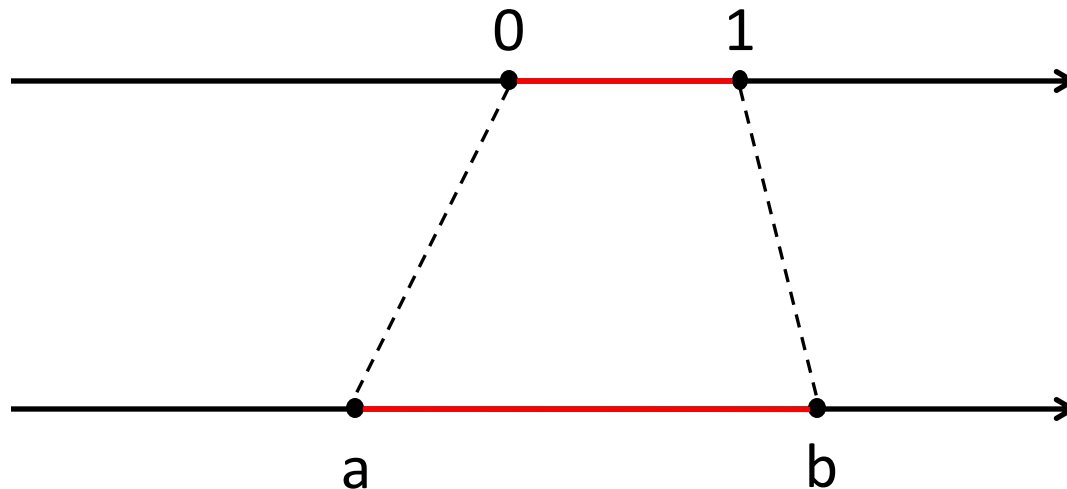
- Generate $U \sim \text{Unif}(0, 1)$
- Output $a + (b - a)U$

Generating Uniform RV

- Let $X \sim \text{Unif}(a, b)$
- If $a \leq x \leq b$, $P(X \leq x) = (x-a)/(b-a)$
- If $x < a$, $P(X \leq x) = 0$; if $x > b$, $P(X \leq x) = 1$
- Thus $G(u) = (b-a)u + a$ for $0 < u < 1$
- Thus, to generate $\text{Unif}(a, b)$, we generate $U \sim \text{Unif}(0, 1)$ and output $X = (b-a)U + a$

Generating Uniform RV

- Let $X \sim \text{Unif}(a, b)$
- If $a \leq x \leq b$, $P(X \leq x) = (x-a)/(b-a)$
- If $x < a$, $P(X \leq x) = 0$; if $x > b$, $P(X \leq x) = 1$
- Thus $G(u) = (b-a)u + a$ for $0 < u < 1$
- Thus, to generate $\text{Unif}(a, b)$, we generate $U \sim \text{Unif}(0, 1)$ and output $X = (b-a)U + a$
- This matches the intuition that a linear transformation of a $\text{Unif}(0, 1)$ should also be uniform



Generating Exponential RV

- Let $X \sim \text{Exp}(\lambda)$
- The pdf $f(x)$ is given by

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

- The cdf $F(x)$ is given by

$$F(x) = 1 - e^{-\lambda x} \text{ for } x > 0 \text{ and } 0 \text{ otherwise}$$

Trick:

$$u = 1 - e^{-\lambda x}$$

$$\Rightarrow x = -\frac{1}{\lambda} \log(1 - u)$$

Pseudo-code:

- Generate $u \sim \text{Unif}(0,1)$

- Output $-\frac{1}{\lambda} \log(1 - u)$

Generating Exponential RV

- Let $X \sim \text{Exp}(\lambda)$
- The pdf $f(x)$ is given by

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

- The cdf $F(x)$ is given by

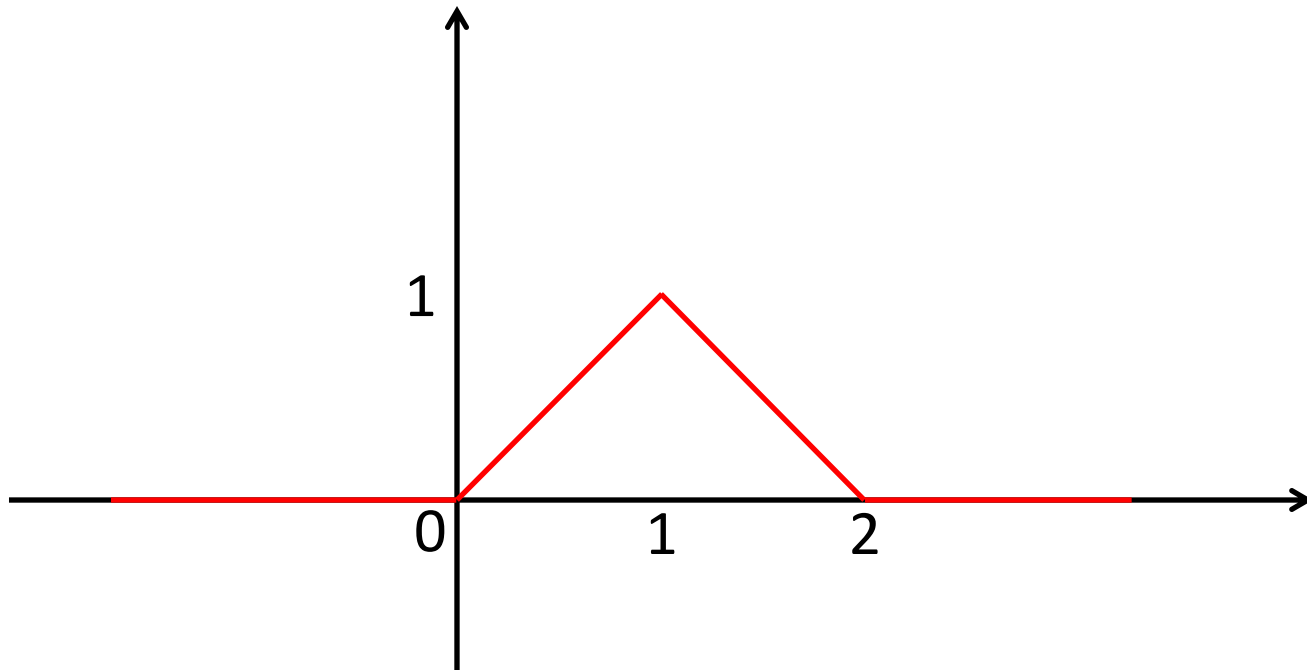
$$F(x) = 1 - e^{-\lambda x} \text{ for } x > 0 \text{ and } 0 \text{ otherwise}$$

- To use the inverse transform method, find the inverse function of F , which is $G(u) = -\log(1-u)/\lambda$
- Thus to generate X :
 - Generate $U \sim \text{Unif}(0, 1)$
 - Set $X = -\log(1-U)/\lambda$

Generating General Continuous RV

- **Example** (triangular density). Generate X with pdf given by

$$f(x) = \begin{cases} x & \text{if } x \in [0, 1] \\ 2 - x & \text{if } x \in [1, 2] \\ 0 & \text{otherwise} \end{cases}$$



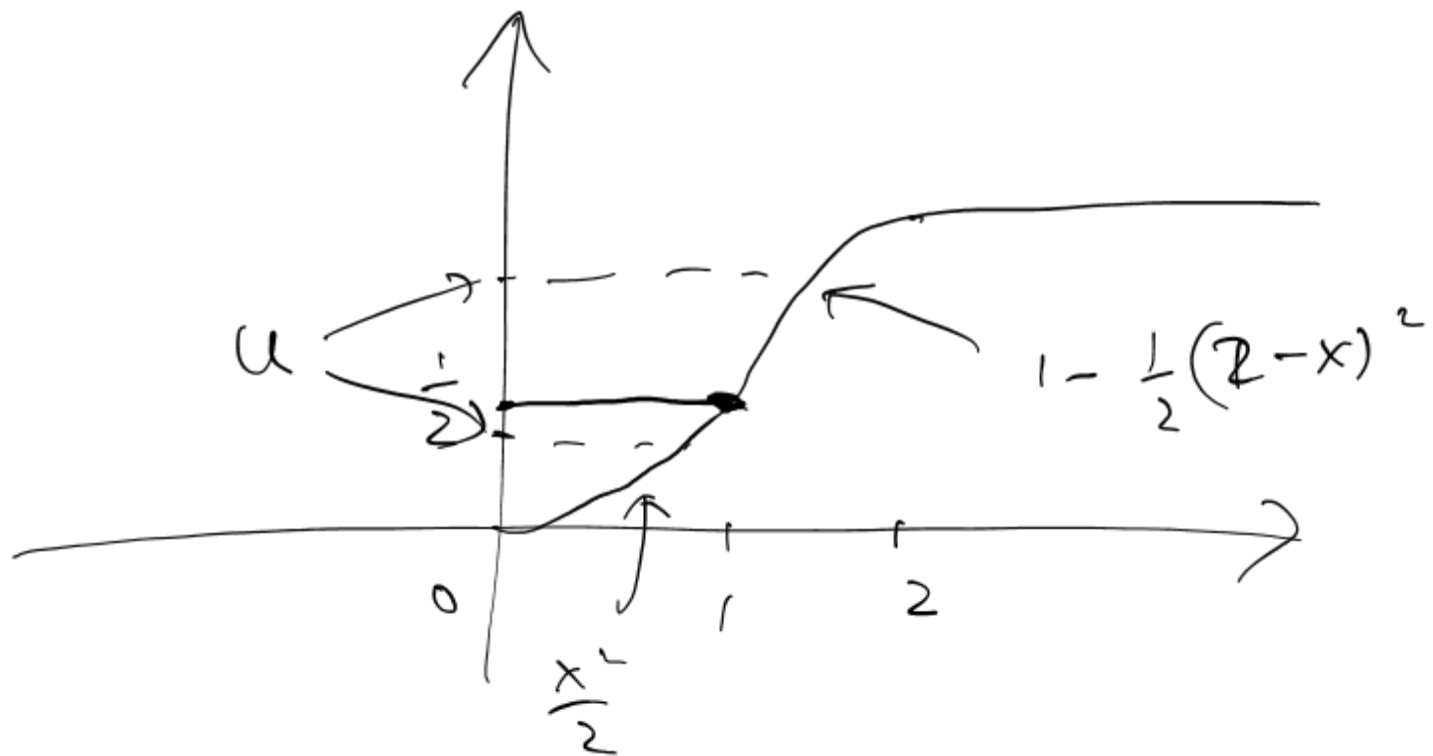
cdf:

$$F(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ \int_0^x \omega \, d\omega & \text{for } 0 < x \leq 1 \\ = \frac{1}{2} \omega^2 \Big|_0^x = \frac{x^2}{2} & \\ \int_0^1 \omega \, d\omega + \int_1^x (2 - \omega) \, d\omega & \text{for } 1 < x \leq 2 \\ = \frac{1}{2} + \frac{1}{2} (2 - \omega)^2 \Big|_x^1 = 1 - \frac{1}{2} (2 - x)^2 & \\ 1 & \text{for } x > 2 \end{cases}$$

$$F(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ \frac{x^2}{2} & \text{for } 0 < x \leq 1 \leftarrow \text{when } x=1, = \frac{1}{2} \\ 1 - \frac{1}{2}(2-x)^2 & \text{for } 1 < x \leq 2 \\ 1 & \text{for } x > 2 \end{cases}$$

Set $u = \frac{x^2}{2} \Rightarrow x = \sqrt{2u}$

Set $u = 1 - \frac{1}{2}(2-x)^2 \Rightarrow x = 2 - \sqrt{2(1-u)}$



Generate $u \sim \text{Unif}(0, 1)$

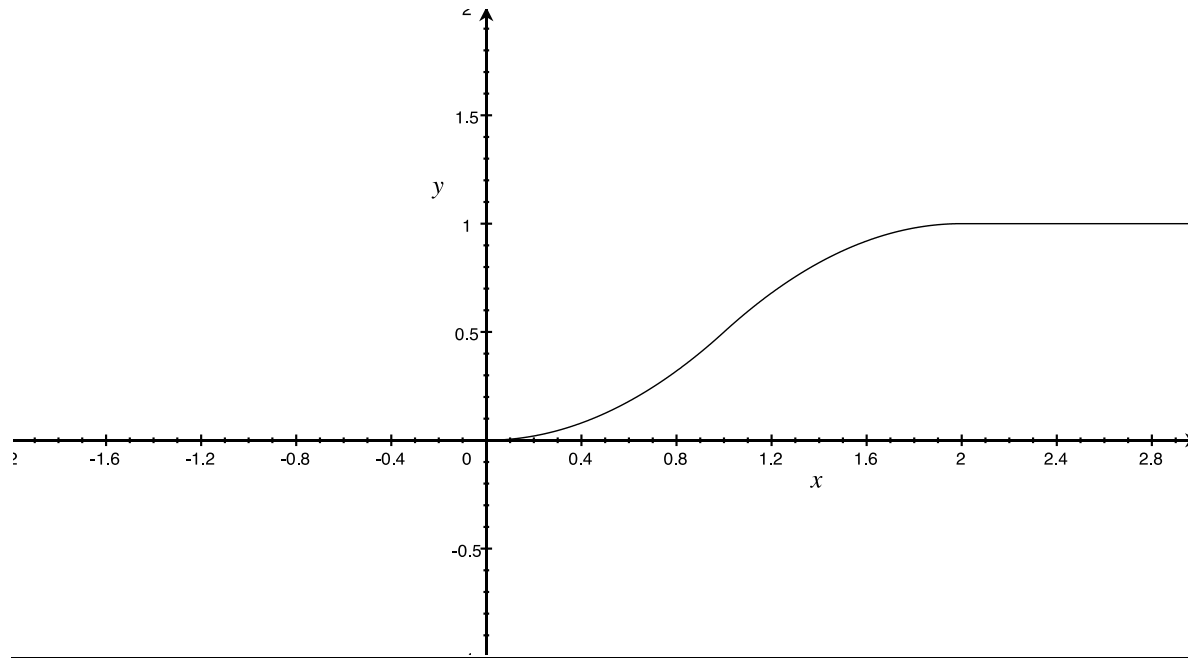
If $u \leq \frac{1}{2}$, output $x = \sqrt{2u}$

If $u > \frac{1}{2}$, output $x = 2 - \sqrt{2(1-u)}$

Generating General Continuous RV

- To apply the inverse transform method, first compute the cdf $F(x)$

$$F(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ \frac{x^2}{2} & \text{if } x \in [0, 1] \\ -\frac{(2-x)^2}{2} + 1 & \text{if } x \in [1, 2] \\ 1 & \text{if } x \geq 2 \end{cases}$$



Generating General Continuous RV

- The inverse function is obtained as follows
- Set $u = x^2/2$; then $x = \sqrt{2u}$
- Set $u = -(2-x)^2/2 + 1$; then $x = 2 - \sqrt{2(1-u)}$
- Thus the inverse function $G(u)$ is

$$G(u) = \begin{cases} \sqrt{2u} & \text{if } u \in [0, 0.5]; \\ 2 - \sqrt{2(1-u)} & \text{if } u \in [0.5, 1] \end{cases}$$

- To generate X with the triangular density
 - Generate $U \sim \text{Unif}(0, 1)$
 - If $U < 0.5$, let $X = \sqrt{2U}$
 - If $U \geq 0.5$, let $X = 2 - \sqrt{2(1-u)}$

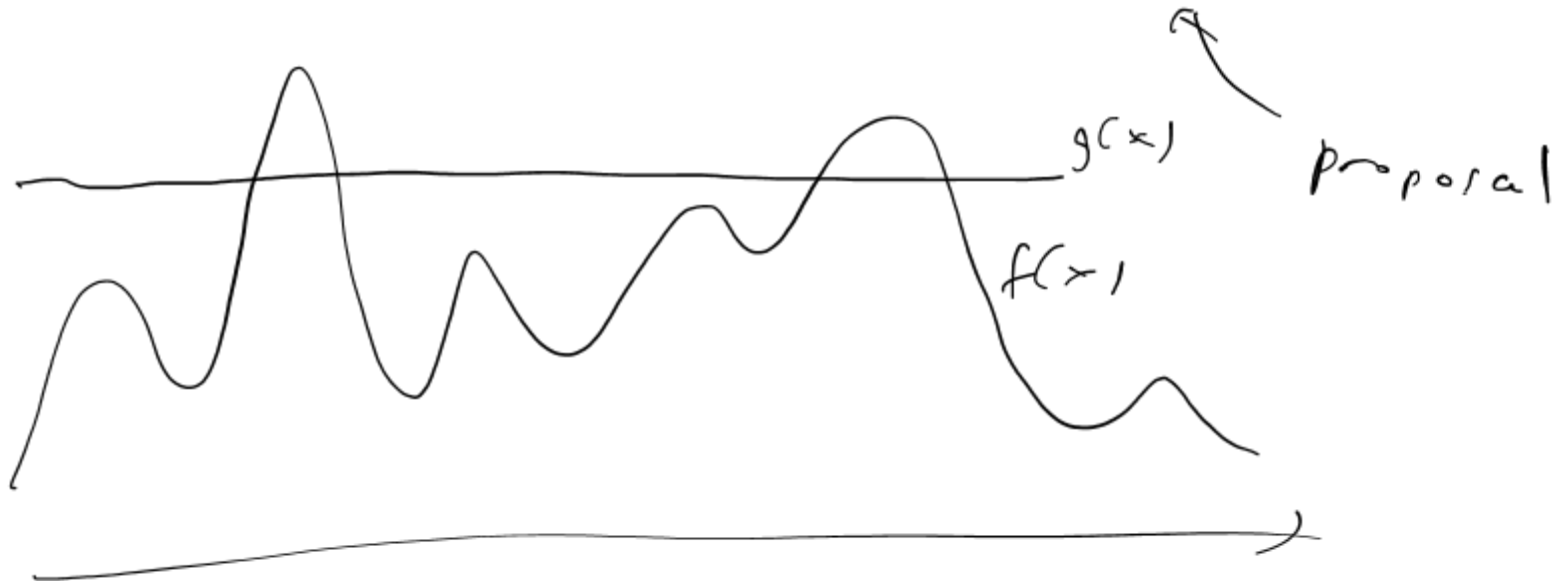
Limitations and Applications

- The inverse transform method applies as long as we can compute the cdf $F(x)$ (and its inverse)
- Not always possible to do; e.g., the normal density function
- For standard distributions, many software packages are available, but some probability distributions may not fit into any existing distributions, in which case the inverse transform method will be useful

Acceptance Rejection Method

A method to generate “difficult” r.v. from “easy” ones.

- We want to generate a r.v. with density $f(x)$ (difficult)
- We can generate a r.v. with density $g(x)$ (easy)



Acceptance Rejection Method

Acceptance rejection method:

- Find a constant C such that

$$\frac{f(x)}{g(x)} \leq C \text{ for all possible } x$$

- To generate one output,

- Generate $Y \sim g(y)$

- “Accept” Y with probability $\frac{f(Y)}{Cg(Y)}$; otherwise “reject” and go back to the previous step

Output the “accepted” Y

- The r.v. from $g(x)$ is called the proposal r.v.

*all x in the
“support” of $f(x)$
(i.e. $f(x) > 0$)*

easily doable

Acceptance Rejection Method

Acceptance rejection method:

- Find a constant C such that

$$\frac{f(x)}{g(x)} \leq C \text{ for all possible } x$$

- To generate one output,
 - Generate $Y \sim g(y)$
 - Generate $U \sim Unif(0,1)$
 - If $U \leq \frac{f(Y)}{Cg(Y)}$, set $X = Y$ (accept); otherwise, go back to the previous step (reject)
 - Output X

using ITM for Ber



Example

- Generate a r.v. with pdf

$$f(x) = 3x^2, \quad 0 \leq x \leq 1$$

- Use a *Unif*(0,1) proposal

$$g(x) = 1, \quad 0 \leq x \leq 1$$

A-R :

Find a constant C such that

$$\frac{f(x)}{g(x)} \leq C$$

for all possible values of x .

$$\Rightarrow \frac{3x^2}{1} \leq C \quad \text{for } 0 \leq x \leq 1$$

We can choose $C = 3$

(or any number ≥ 3)

① Generate $Y \sim g(y) = \text{Unif}(0, 1)$

② "Accept" Y with prob $\frac{f(y)}{Cg(y)} = \frac{3y^2}{3 \cdot 1} = y^2$,

otherwise "reject" and go back to ①

pseudo code!

- ① Generate $Y \sim \text{Unif}(0, 1)$
- ② Generate $U \sim \text{Unif}(0, 1)$
- ③ If $U \leq Y^2$, set $X = Y$;
otherwise go back to ①
- ④ Output X .

Example

- Generate a r.v. with pdf

$$f(x) = 3x^2, \quad 0 \leq x \leq 1$$

- Use a *Unif*(0,1) proposal

$$g(x) = 1, \quad 0 \leq x \leq 1$$

- We have

$$\frac{f(x)}{g(x)} = \frac{3x^2}{1} = 3x^2 \leq 3 \text{ for any } 0 \leq x \leq 1$$

- Thus we can choose $C = 3$

Example

Acceptance-rejection method:

- Generate $Y \sim Unif(0,1)$
- Generate $U \sim Unif(0,1)$
- If $U \leq \frac{f(Y)}{cg(Y)} = Y^2$, set $X = Y$ (accept); otherwise, go back to the previous step (reject)
- Output X

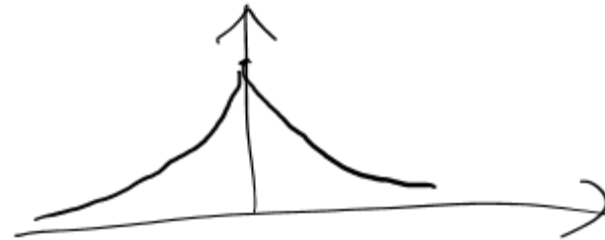
Example

- Generate the standard normal r.v.

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}, \quad -\infty < x < \infty$$

- Use a proposal

$$g(x) = \frac{1}{2} e^{-|x|}, \quad -\infty < x < \infty$$



↑
“two-sided” exponential variate

A-R:

Find a constant C such that

$$\frac{f(x)}{g(x)} \leq C \quad \text{for all possible } x$$

$$\frac{\frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}}{\frac{1}{2} e^{-|x|}} \leq C \quad \text{for } -\infty < x < \infty$$

$$C = \max_{-\infty < x < \infty} \int \frac{2}{\pi} e^{-\frac{x^2}{2} + |x|}$$

$$= \int \frac{2}{\pi} e^{\max_{-\infty < x < \infty} \left\{ -\frac{x^2}{2} + |x| \right\}}$$

$$= \int \frac{2}{\pi} e^{\max_{x \geq 0} \left\{ -\frac{x^2}{2} + x \right\}}$$

$$= \left(\frac{2}{\pi} \right) e^{-\frac{1}{2} + 1} = \int \frac{2}{\pi} e^{\frac{1}{2}} = \sqrt{\frac{2e}{\pi}}$$



Take derivative
on $-\frac{x^2}{2} + x$ and
set to 0:
 $-x + 1 = 0$

$$\Rightarrow x = 1$$

① Generate $Y \sim g(y) = \frac{1}{2} e^{-|y|}$

② "Accept" Y w.p. $\frac{f(Y)}{cg(Y)} = \frac{\frac{1}{\sqrt{2\pi}} e^{-\frac{Y^2}{2}}}{\int \frac{2e}{\pi 2} e^{-|y|}}$

$$= e^{-\frac{Y^2}{2} + |Y| - \frac{1}{2}}$$

otherwise "reject" and go back to ①

Output "accepted" Y .

Pseudo code:

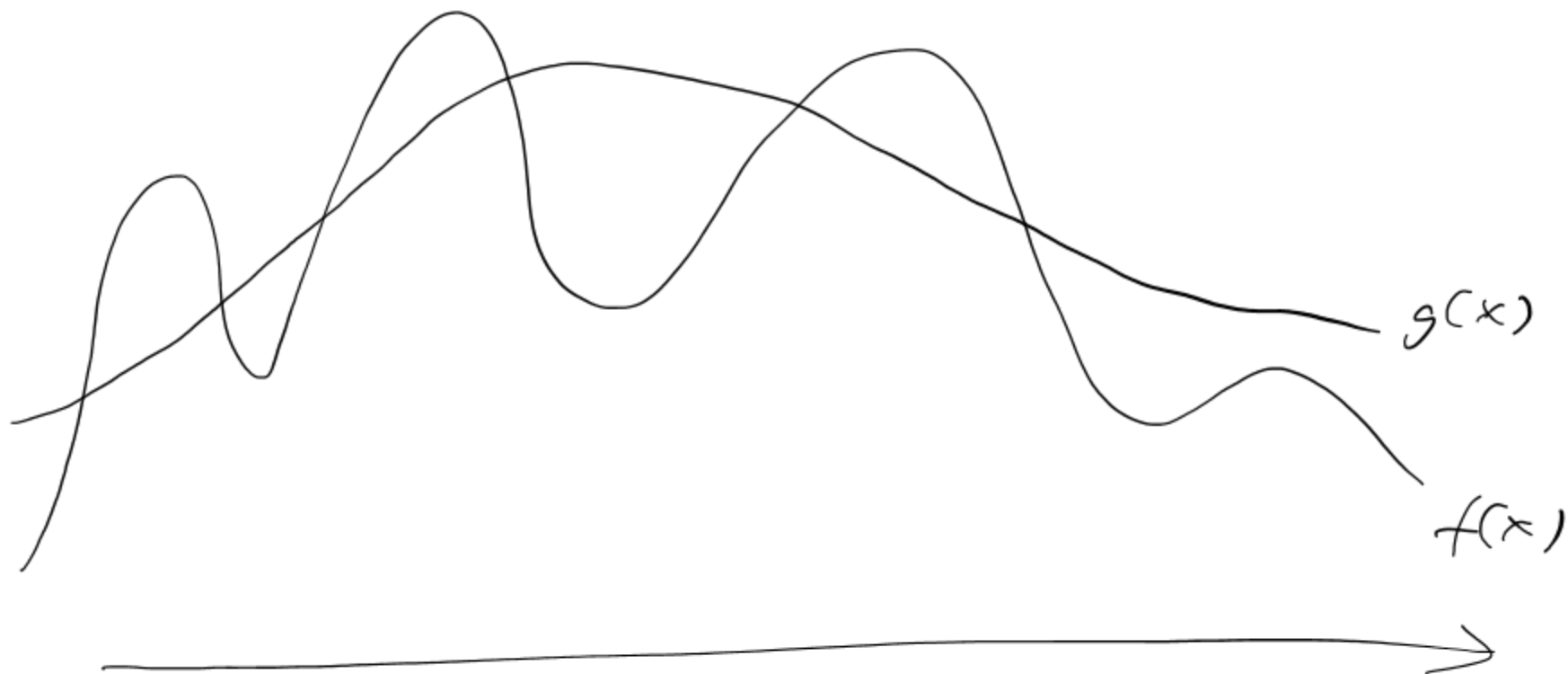
- ① Generate $Y \sim \frac{1}{2} e^{-|y|}$
- ② Generate $U \sim \text{Unif}(0, 1)$
- ③ If $U \leq e^{-\frac{y^2}{2} + |y| - \frac{1}{2}}$, set $x = y$;
otherwise go back to ①
- ④ Output x .

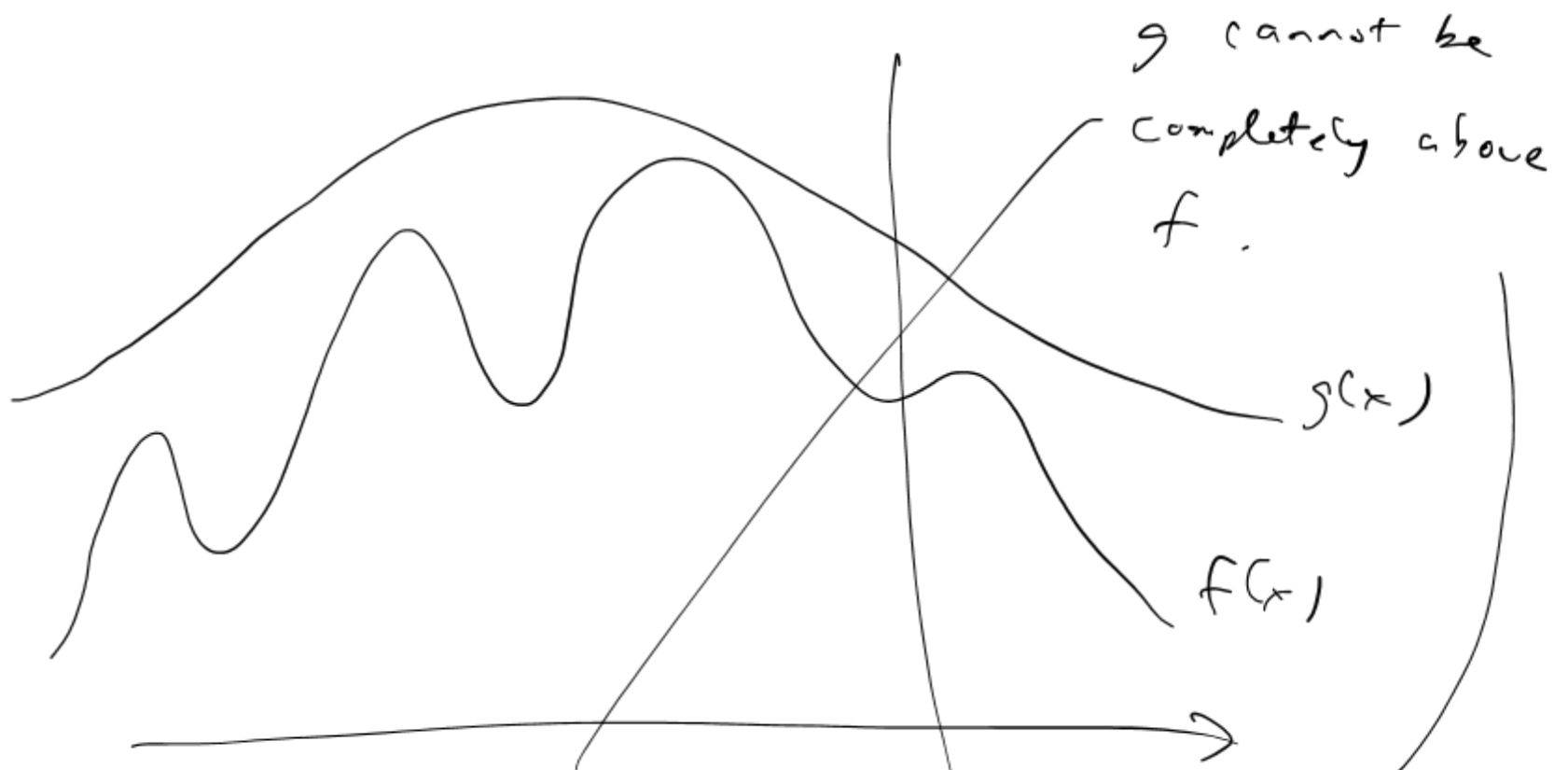
Note: To generate $Y \sim \frac{1}{2} e^{-|Y|}$

Approach 1: Use (TM)

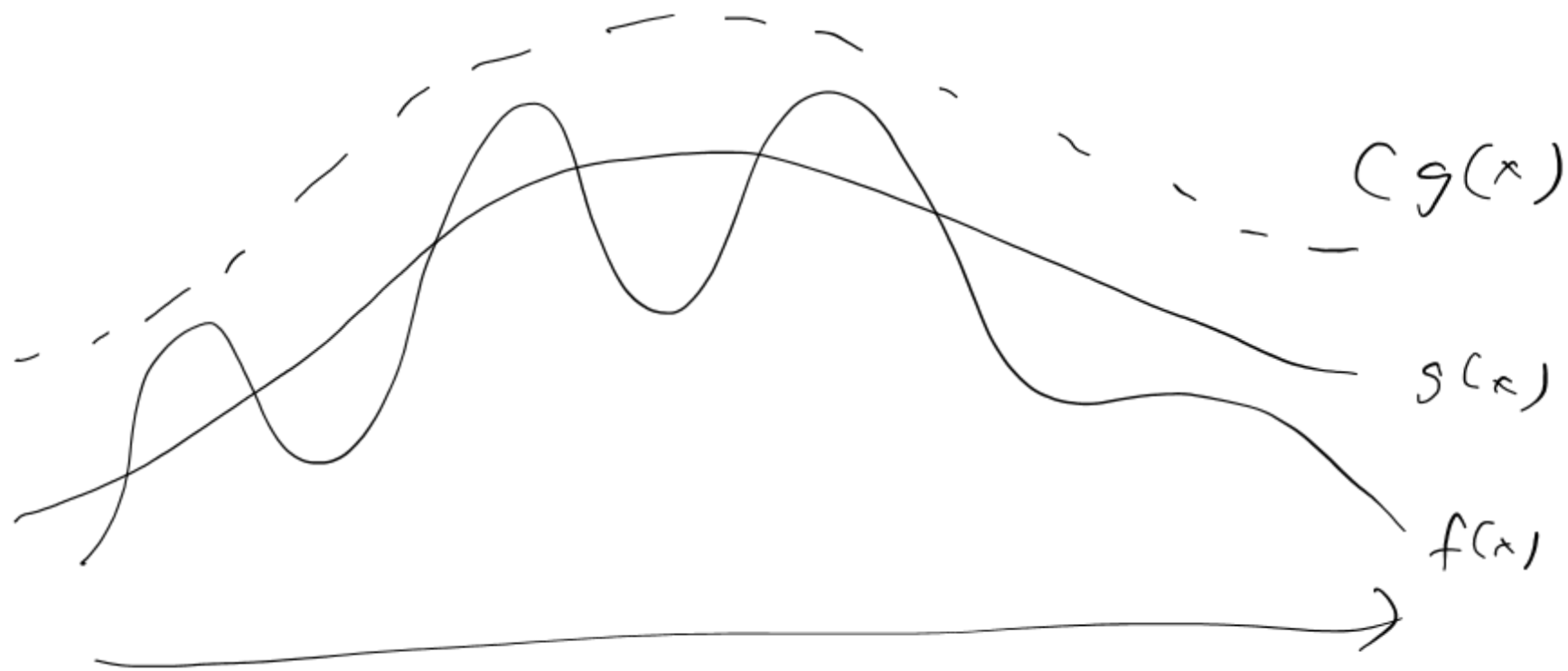
Approach 2:

- Generate a "sign variable":
 W takes ± 1 and -1 w.p. $\frac{1}{2}$.
- Generate $V \sim \text{Exp}(1)$ (use (TM))
- Output $W \cdot V$.





If this happens, then one of them
have area under curve $\neq 1 \Rightarrow$ either f or g is not
valid pdf



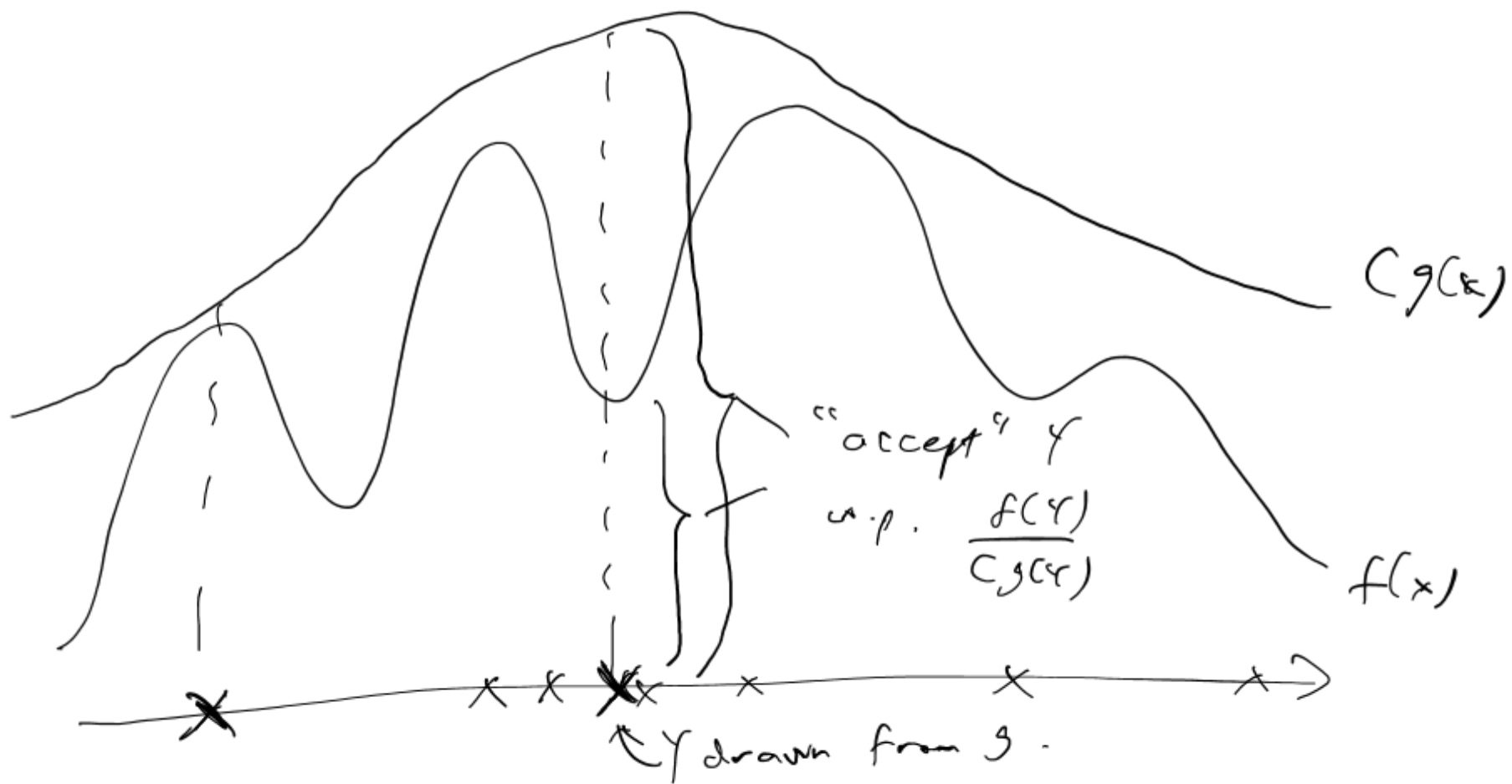
Choose C such that $Cg(x)$ completely lies above $f(x)$

This C is the constant in the
A-R algorithm, because

$$Cg(x) \geq f(x) \text{ means}$$

$$\frac{f(x)}{g(x)} \leq C$$

From our observation, C has to be ≥ 1 .
 C can be exactly 1 if $f=g$.



The "acceptance" mechanism adjusts
sample, drawn from g to act more
like samples drawn from f . That is,
if $C(g(x)) \approx f(x)$, accept y with
high prob.; if not, accept with low
prob.

Efficiency

- The efficiency of the algorithm depends on the acceptance rate, i.e., the more frequent that Y is accepted, the faster is the algorithm
- The number of trials to get one acceptance is $\text{Geo}(p)$, where p is the acceptance probability
- The acceptance probability is $p=1/C$, where C is the constant that bounds $f(x)/g(x)$
- If $g(x)$ is close to $f(x)$, then C is small, so p is large

- runtime of A-R algorithm is random

- concrete measurement of efficiency is
expected runtime

- runtime = # iterations (or generation of Y)
to obtain an acceptance

$$\sim \text{Geo}(p)$$

$$p = \frac{1}{C}$$

Intuitively, when C is small, the acceptance probability (unconditional) is large

E.g. when $C = 1$ (as small as possible),

$f = g$, and acceptance prob = 1.

$$\text{expected runtime} = \frac{1}{p} = C$$

The lower the C , the faster is the
A-R algo.

When g is exactly f , C is the lowest ($=1$)
(this is useful to know in "theory").

The "theory" gives the guideline that a good proposal g should be as close to f as possible while at the same time easy to simulate from.

